

Code Explanation

Sample Data Generator for SFDC Account ETL Pipeline

This code generates sample data for testing the SFDC Account ETL pipeline. It creates a small dataset that mimics the structure of the source data.

The code defines a function `generate_sample_data` that takes a `SparkSession` object and the number of rows to generate as input, and returns a `DataFrame` with the sample data. The data is generated based on a predefined schema that matches a subset of the source table.

Overview of the Code

The code is a Python module that uses the PySpark library to generate sample data for testing the SFDC Account ETL pipeline. It defines a function to generate the sample data and a main block to run the function and display the generated data.

Step 1: Importing Necessary Libraries

The code starts by importing the necessary libraries, including `datetime`, `random`, `decimal`, and PySpark libraries.

```
from datetime import datetime, timedelta
import random
from decimal import Decimal

from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, DecimalType, DateType
```

Step 2: Defining the `generate_sample_data` Function

The `generate_sample_data` function takes a `SparkSession` object and the number of rows to generate as input, and returns a `DataFrame` with the sample data.

```
def generate_sample_data(spark: SparkSession, num_rows: int = 10):
    """
    Generate sample data for testing the ETL pipeline.

    Args:
        spark: SparkSession object
        num_rows: Number of sample rows to generate

    Returns:
        DataFrame with sample data
    """
```

Step 3: Defining the Schema for the Sample Data

The schema is defined using the `StructType` and `StructField` classes from PySpark. The schema includes fields such as `SK_ACCOUNT_ID`, `DW_SF_ACCOUNT_ID`, `DW_SF_USER_ID`, and others.

```

schema = StructType([
    StructField("SK_ACCOUNT_ID", DecimalType(15, 0), False),
    StructField("DW_SF_ACCOUNT_ID", DecimalType(15, 0), False),
    StructField("DW_SF_USER_ID", DecimalType(15, 0), False),
    StructField("DW_SF_CUSTOMER_TYPE_ID", DecimalType(15, 0), False)
),
    StructField("SOURCEKEYID", StringType(), True),
    StructField("DELETED_FLG", StringType(), True),
    StructField("ACC_NAME", StringType(), True),
    StructField("ACC_PARTNER_REGION", StringType(), True),
    StructField("ACC_ACCOUNT_NUMBER", StringType(), True),
    StructField("ACC_PROSPECT_NUMBER", StringType(), True),
    StructField("ACC_BILLING_POSTAL_CODE", DecimalType(15, 0), True)
),
    StructField("ACC_SHIPPING_POSTAL_CODE", DecimalType(15, 0), True)
),
    StructField("LATITUDE", DecimalType(18, 15), True),
    StructField("LONGITUDE", DecimalType(18, 15), True),
    StructField("COMPANY_DESCRIPTION", StringType(), True),
    StructField("TAG_ACCOUNT_AS", StringType(), True),
    StructField("ACC_OPEN_OPP_DOLLARS_CURNCY_FY", DecimalType(18, 3)
, True),
    StructField("SRC_ACC_CREATED_DATE", DateType(), True),
    StructField("DW_UPDATE_DT", DateType(), True),
    StructField("SUSPENDED_STATE", StringType(), True),
])

```

Step 4: Generating the Sample Data

The sample data is generated using a loop that creates a list of rows based on the defined schema. The data is generated randomly using various techniques such as using random numbers, dates, and strings.

```

data = []
regions = ["NAMR", "EMEA", "APAC", "LATAM"]
base_date = datetime.now() - timedelta(days=30)

for i in range(1, num_rows + 1):
    account_id = 1000000000000 + i
    region = regions[i % len(regions)]
    update_dt = base_date + timedelta(days=i % 20)

    row = (
        Decimal(account_id),                # SK_ACCOUNT_ID
        Decimal(20000000000000 + i),       # DW_SF_ACCOUNT_ID
        Decimal(30000000000000 + i),       # DW_SF_USER_ID
        Decimal(random.randint(1, 5)),      # DW_SF_CUSTOMER_T
        f"SRC-{i:05d}",                     # SOURCEKEYID
        "N",                                # DELETED_FLG
        f"Test Account {i}",                # ACC_NAME
        region,                             # ACC_PARTNER_REG
        f"ACC{i:06d}",                       # ACC_ACCOUNT_NUM
        f"PROS{i:06d}",                      # ACC_PROSPECT_NU
        Decimal(random.randint(10000, 99999)), # ACC BILLING POS
    )
    data.append(row)

```

```

TAL_CODE
    Decimal(random.randint(10000, 99999)),      # ACC_SHIPPING_PO
STAL_CODE
    Decimal(f"{random.uniform(30, 50):.15f}"), # LATITUDE
    Decimal(f"{random.uniform(-120, -70):.15f}"), # LONGITUDE
    f"This is a sample company description for account {i}. " *
5, # COMPANY_DESCRIPTION
    f"tag1,tag2,tag3,tag{i}",                    # TAG_ACCOUNT_AS
    Decimal(random.uniform(1000, 100000)),      # ACC_OPEN_OPP_DO
LLARS_CURNCY_FY
    base_date - timedelta(days=random.randint(30, 365)), # SRC_A
CC_CREATED_DATE
    update_dt,                                  # DW_UPDATE_DT
    "ACTIVE" if i % 5 != 0 else "SUSPENDED",    # SUSPENDED_STATE
)
data.append(row)

```

Step 5: Creating a DataFrame from the Sample Data

The sample data is used to create a DataFrame using the `createDataFrame` method of the SparkSession object.

```

df = spark.createDataFrame(data, schema)
return df

```

Step 6: Running the `generate\_sample\_data` Function

The `generate\_sample\_data` function is run in the main block of the code, which creates a SparkSession object, generates sample data, and displays the generated data.

```

if __name__ == "__main__":
    spark = SparkSession.builder
        .appName("Generate Sample Data")
        .master("local[*]")
        .getOrCreate()

    sample_df = generate_sample_data(spark, 20)
    print(f"Generated {sample_df.count()} sample rows")
    sample_df.show(5, truncate=False)

    sample_df.write.mode("overwrite").csv("sample_data.csv", header=
True)

    spark.stop()

```